Partial Differential Equations 3 (Numerical)
Lecture 2 - Academic Year 2025/2026

Prof Halim Kusumaatmaja
Email: halim.kusumaatmaja@ed.ac.uk

Semester 2, Edinburgh Engineering

Aims and Objectives

1. To discuss how a PDE can be discretised.
2. To describe Jacobi and Gauss-Sidel iteration algorithms.
3. To discuss common boundary conditions.
4. To distringuish validation vs verification.

Laplace equation in discretised form

In Lecture 1 we have seen that the Laplace equation is an elliptic PDE. Analytically, the Laplace equation in 2D is given by

$$u_{xx} + u_{yy} = 0. \tag{1}$$

We now want to write the Laplace equation in its discretised form. For simplicity, we have assumed we will employ a rectangular grid with spacing Δx and Δy, as illustrated in Fig. 1.

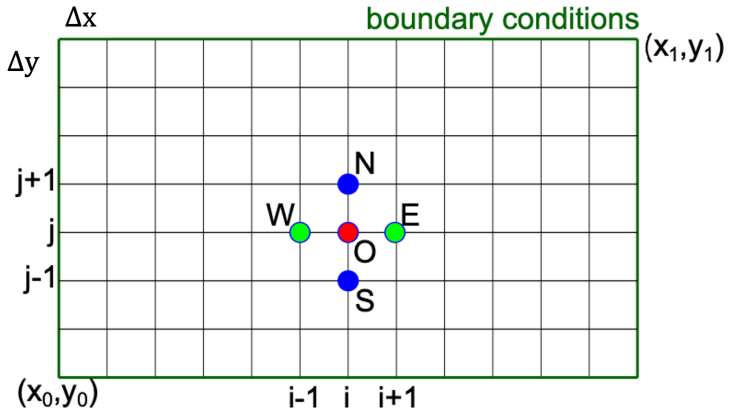

Figure 1: Illustration of a rectangular grid.

In discrete form, using second order accurate scheme for u_{xx} and u_{yy} we discussed in lecture 1, the Laplace equation becomes

$$u_{xx} + u_{yy} = \frac{u_E - 2u_O + u_W}{\Delta x^2} + \frac{u_N - 2u_O + u_S}{\Delta y^2} = 0, \tag{2}$$

which we can rearrange to write

$$u_O = \frac{1}{2(1+\beta^2)}(u_E + u_W + \beta^2(u_N + u_S)) \tag{3}$$

with $\beta = \frac{\Delta x}{\Delta y}$. In most cases we will employ $\Delta x = \Delta y$ such that $\beta = 1$.

Jacobi iteration

It is useful at this point to take a step back and consider Eq. (3). This relation between the five grid points denoted by u_O, u_E, u_W, u_N, and u_S is only correct when the Laplace equation is satisfied. A priori we do not know what the values for these u's are as they are precisely what we are trying to solve.

The key idea behind the Jacobi iteration algorithm is that we can exploit the PDE of interest in its discretised form, and if we iterate the algorithm enough times, we will converge to the correct solution subject to the boundary conditions of the problem. Illustrating this with the Laplace equation, the main equation that we will compute iteratively is

$$u_{i,j}^{m+1} = \frac{1}{2(1+\beta^2)}(u_{i+1,j}^{m} + u_{i-1,j}^{m} + \beta^2(u_{i,j+1}^{m} + u_{i,j-1}^{m})) \tag{4}$$

where m is the iteration counter from $m = 0$ to $m \to \infty$. In practise, of course we cannot run the iterative scheme infinitely. We stop when the solution has achieved a defined tolerance criterion. This can be based on comparison to an analytical solution if known, or the rate of change of the numerical solution. For convenience, we have also slightly rewritten the indices using the i, j notation. You should convince yourself that this is correct.

A word of caution: It is worth mentioning that we (the community) use the subscripts for different things. In Lecture 1, when we discussed the analytical form of the PDEs, the subscript usually corresponds to the partial derivative. When we write equations in lattice / discretised form, the subscript is often used to index/label which grid point we refer to. So be careful, we need to understand the context it is used for.

We can now write the pseudocode for Jacobi iteration when applied to the 2D Laplace equation as follows:

set $m = 0$
initialise $u_{ij}^{m=0}$ *for all* i, j*; this is your best or simplest guess usually*
compute $\beta = \Delta x/\Delta y$
repeat
    apply boundary conditions
    compute Eq. (4) *for all* i, j
    increment m
until max $|u_{ij}^{m} - u_{ij}^{m-1}| < \epsilon$*; or any other stop tolerance criterion*

You will implement Jacobi iteration in computing workshop 2, and hence it is important that you understand the algorithm.

Please now look at the online interactive tool (see link below under Additional Materials). Click on Jacobi Iteration and look at the section on "Effect of Number of Iterations". You can visualise some steps of Jacobi iteration for a coarse grid, as illustrated in Fig. 2. In this example, we initialise the calculation with $u(x, y) = 0$ everywhere except for the boundary. Can you rationalise what you observe based on the Jacobi iteration algorithm?

Gauss-Sidel iteration

When we carry out the Jacobi iteration, it is useful to reflect that we usually update the $u_{i,j}$'s value one grid point at a time, for example by running a 'for loop' on indices i and j. This means that before we calculate $u_{i,j}^{m+1}$ actually we already know the updated value for $u_{i-1,j}^{m+1}$ and $u_{i,j-1}^{m+1}$. Assuming the $m + 1$ iteration values are better than at m iteration, the key idea of the Gauss-Sidel algorithm is to take advantage of the latest updates as much as possible. Hence, the Gauss-Sidel iteration scheme is similar to Jacobi iteration except we exploit $m + 1$ iteration values (highlighted in blue below) when they are already available,

$$u_{i,j}^{m+1} = \frac{1}{2(1+\beta^2)}(u_{i+1,j}^{m} + u_{i-1,j}^{m+1} + \beta^2(u_{i,j+1}^{m} + u_{i,j-1}^{m+1})). \tag{5}$$

In terms of performance, the improvement for Gauss-Sidel iteration is modest (or can be worse in terms of computational time if not implemented carefully) compared to Jacobi iteration.

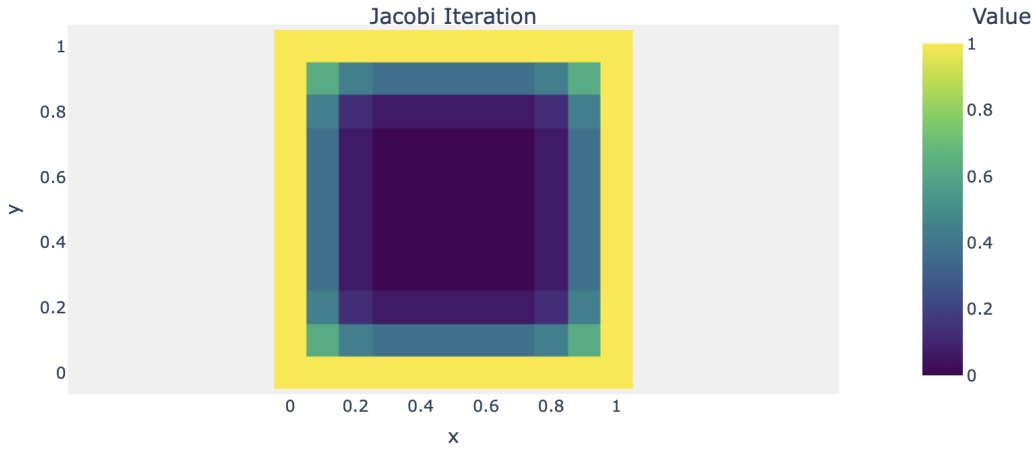

Figure 2: Illustration of Jacobi Iteration for a simple example where we initialise the calculation with $u(x, y) = 0$ everywhere except for the boundary.

Exercise 1: Consider the pseudocode we had above for the Jacobi Iteration. What change(s) do we need to make for the Gauss-Sidel iteration?

Boundary Condition

Recall Fig. 1 where we show a typical grid that we will employ in our computations. The grid points that sit on the boundary of the computational domain (highlighted in green in the figure) are boundary nodes and they need special treatment which we often call as the boundary condition. In particular there are three types of boundary conditions that appear regularly in numerical PDEs:

- Periodic boundary condition: this is when we "wrap around" the grid such that the west neighbour of the left edge of the grid is the right edge, the east neighbour of the right edge is the left edge, and similarly for the top and bottom edges of the grid.
- Neumann boundary condition: this is when the normal derivative of the solution is fixed at the boundary. For example, consider a 1-D problem with a Neumann boundary condition $u_x = a$ at the right boundary. In terms of numerical methods, one popular way to implement this is to exploit our central difference scheme to set the outer boundary to have the following value at every iteration:

$$u_x = \frac{u_{i+1} - u_{i-1}}{2\Delta x} = a \quad \rightarrow u_{i+1} = u_{i-1} + (2\Delta x)a. \tag{6}$$

- Dirichlet boundary condition: this is when the value of the solution is fixed at the boundary. Here we just set the specified value at every iteration on the boundary.

Example Problem

Let us consider an example where we will use the Jacobi iteration to solve a Laplace equation on a grid defined by $0 \leq x \leq 2$ and $0 \leq y \leq 1$ subject to the following boundary condition:

$$\begin{aligned} u(x, 0) &= 0 \quad \text{for } 0 \leq x \leq 2 \\ u(x, 1) &= 0 \quad \text{for } 0 \leq x \leq 2 \\ u(0, y) &= 0 \quad \text{for } 0 \leq y \leq 1 \\ u(2, y) &= \sin(2\pi y) \quad \text{for } 0 \leq y \leq 1 \end{aligned}$$

This problem has an analytical solution $u(x, y) = \sin(2\pi y)\frac{\sinh 2\pi x}{\sinh 2\pi}$, which we visualise in Fig. 4.

Exercise 2: In the interactive tool, see section "Effect of Mesh Spacing", you can compare the converged solution as a function of the resolution. Given the results shown, which grid size would you use for this computation? Rationalise your choise.

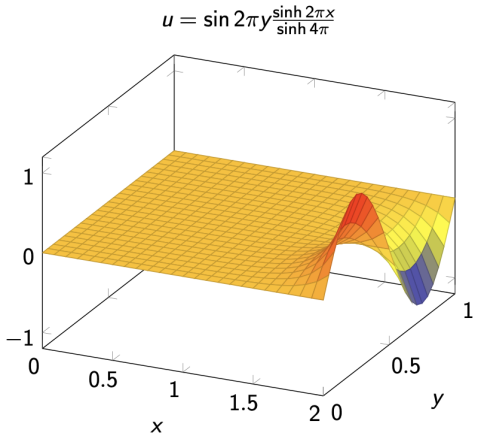

Figure 3: Analytical solution to the example problem considered.

Validation vs Verification

Once we implement our numerical scheme and get some solutions, we need to consider quality assurance. We distinguish two different approaches: verification and validation, see Fig. **??**.

Verification is about **solving the equations right**. It is a process of determining that a model implementation accurately represents our conceptual description of the model and the solution to the model. This typically involves checking the order of accuracy, checking the mesh is sufficiently fine, and quantifying truncation errors.

Validation is about **solving the right equations**. It is a process of determining the degree to which a model is an accurate representation of the real world from the perspective of the intended use of the model. We can compare with experimental results and/or other types of theoretical/numerical results.

An example: we may solve some fluid flow problem with the Euler equation. We can validate if the Euler equation is suitable. It may not be if viscous dissipation is important. At the same time, we can verify if our numerical solver based on the Euler equation is implemented correctly and that we have sufficient accuracy.

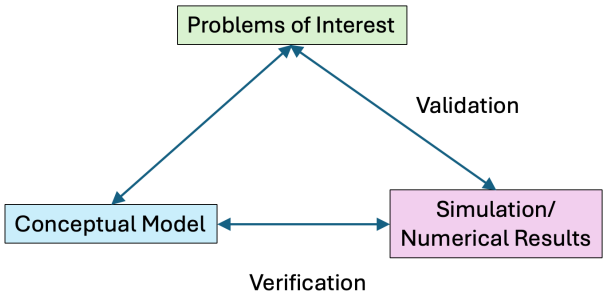

Figure 4: Verification vs Validation in the context of numerical PDEs.

Additional Materials

- Interactive tool: `https://kusumaatmaja.github.io/PDE3/examples`
- For more details on verification and validation: Roache, P.J., Verification and Validation in Computational Science and Engineering, Hermosa Publishers, 1998.

Version Control:
- version 1.0 on 20 December 2025